MANUAL DE DESPLIEGUE — AGROGESTIÓN

Índice

1. Introducción
2. Requisitos previos
3. Obtener el código fuente
4. Configurar Supabase (Backend)
   - 4.1. Crear proyecto en Supabase
   - 4.2. Crear las tablas de la base de datos
   - 4.3. Configurar políticas RLS
   - 4.4. Crear el usuario administrador
   - 4.5. Obtener credenciales de conexión
5. Configurar el entorno local
   - 5.1. Instalar Node.js
   - 5.2. Instalar dependencias del proyecto
   - 5.3. Configurar variables de entorno
6. Ejecutar la aplicación en modo desarrollo
7. Construir la aplicación para producción
8. Desplegar en un servidor
   - 8.1. Despliegue en Vercel (recomendado)
   - 8.2. Despliegue en Netlify
   - 8.3. Despliegue en servidor propio (Apache/Nginx)
9. Verificación del despliegue
10. Solución de problemas comunes

---

1. Introducción

Este manual describe paso a paso todo lo necesario para poner en marcha la aplicación Agrogestión, tanto en un entorno de desarrollo local como en un servidor de producción.

La aplicación cuenta con **10 tablas** en PostgreSQL, políticas RLS (Row Level Security) por rol, funciones auxiliares, triggers de notificación automática y soporte para 3 idiomas (español, inglés y rumano).

Está dirigido a desarrolladores o técnicos encargados de instalar, configurar y desplegar la aplicación.

---

2. Requisitos previos

Antes de comenzar, asegúrate de tener instalado lo siguiente:

Software	Versión mínima	Descarga
Node.js	18.x (recomendado 20.x LTS)	https://nodejs.org/
npm	9.x (incluido con Node.js)	(se instala con Node.js)
Git	2.x	https://git-scm.com/
Navegador	Chrome 90+, Firefox 90+	(cualquier navegador moderno)

Verificar las versiones instaladas:

Abre un terminal (CMD, PowerShell o Bash) y ejecuta:

```
node --version
# Debe mostrar v18.x.x o superior

npm --version
# Debe mostrar 9.x.x o superior

git --version
# Debe mostrar git version 2.x.x
```

Si alguno de los comandos no es reconocido, debes instalar el software correspondiente desde los enlaces indicados.

---

3. Obtener el código fuente

Opción A: Clonar desde GitHub

```
git clone https://github.com/Juanfrina/Agrogestion.Web.git
cd Agrogestion.Web
cd AGROGESTION
```

Opción B: Descargar como ZIP

1. Ir a https://github.com/Juanfrina/Agrogestion.Web
2. Pulsar el botón verde **"Code"** → **"Download ZIP"**.
3. Descomprimir el archivo en la ubicación deseada.
4. Abrir un terminal y navegar hasta la carpeta `AGROGESTION/` dentro del proyecto.

---

4. Configurar Supabase (Backend)

4.1. Crear proyecto en Supabase

1. Accede a https://supabase.com y crea una cuenta (puedes usar GitHub para registrarte).
2. En el dashboard, pulsa **"New Project"**.
3. Configura el proyecto:
   - **Nombre del proyecto:** agrogestion
   - **Contraseña de la base de datos:** (elige una contraseña segura y guárdala)
   - **Región:** Elige la más cercana a tu ubicación (ej: West EU - Ireland)
4. Pulsa **"Create new project"** y espera a que se aprovisione (1-2 minutos).

4.2. Crear las tablas de la base de datos

Una vez creado el proyecto, ve a **SQL Editor** en el menú lateral izquierdo de Supabase.

El proyecto incluye el script completo en el archivo `AGROGESTION/src/database/supabase/schema.sql`. Abre dicho archivo, copia **todo** su contenido y pégalo en el SQL Editor de Supabase.

El script crea automáticamente:

Elemento	Cantidad	Descripción
Tablas	10	rol, estados_tarea, perfiles, terreno, tarea, gerente_capataz, tarea_trabajador, capataz_trabajador, comentarios_tarea, notificaciones
Funciones auxiliares	8	get_my_role, is_admin, is_email_taken, handle_new_user, helpers RLS...
Triggers	3	Creación automática de perfil, notificación a capataz, notificación a trabajador
Políticas RLS	~50	Control de acceso por rol para todas las tablas
Inserts de datos iniciales	—	Los roles y estados se insertan desde la aplicación o manualmente

Insertar roles por defecto (ejecutar en SQL Editor tras el schema):

```
INSERT INTO rol (id_rol, nombre, descripcion) VALUES
    (1, 'ADMIN',       'Administrador del sistema'),
    (2, 'GERENTE',     'Gestor de terrenos y tareas'),
    (3, 'CAPATAZ',     'Ejecutor de tareas y coordinador de trabajadores'),
    (4, 'TRABAJADOR',  'Trabajador de apoyo en tareas');
```

Insertar estados de tarea por defecto:

```
INSERT INTO estados_tarea (nombre, descripcion) VALUES
    ('PENDIENTE',    'La tarea ha sido creada pero no asignada.'),
    ('ASIGNADA',     'La tarea ha sido asignada a un capataz.'),
    ('ACEPTADA',     'El capataz ha aceptado la tarea.'),
    ('RECHAZADA',    'El capataz ha rechazado la tarea.'),
    ('EN_PROGRESO',  'La tarea está siendo ejecutada.'),
    ('COMPLETADA',   'La tarea ha sido finalizada correctamente.');
```

Pulsa **"Run"** para ejecutar cada bloque. Deberías ver el mensaje "Success" sin errores.

> ⚠️ ***IMPORTANTE:*** *Si ya tienes tablas de una versión anterior, descomenta y ejecuta primero la* ***Parte 0 (limpieza)*** *del script schema.sql para eliminar las tablas existentes.*

4.3. Configurar políticas RLS

Las políticas RLS (Row Level Security) **ya están incluidas** en el script `schema.sql` que ejecutaste en el paso anterior. No es necesario configurarlas manualmente.

El script habilita RLS en las 10 tablas y crea ~50 políticas que controlan:

Tabla	Políticas principales
perfiles	Cada usuario ve su perfil; admin ve todos; gerente ve sus capataces; capataz ve sus trabajadores
terreno	Gerente gestiona sus terrenos; admin ve todos; capataz/trabajador ven los de sus tareas

tarea	Gerente ve/crea las suyas; capataz ve las asignadas; trabajador ve donde participa
gerente_capataz	Gerente/capataz ven sus relaciones; gerente inserta/elimina
tarea_trabajador	Capataz asigna; trabajador acepta/rechaza; gerente ve las de sus tareas
capataz_trabajador	Capataz gestiona; trabajador ve sus relaciones
comentarios_tarea	Participantes de la tarea pueden ver y crear; solo el autor puede eliminar
notificaciones	Cada usuario solo ve sus propias notificaciones
rol, estados_tarea	Solo lectura para usuarios autenticados

Para verificar que las políticas se crearon correctamente, ve a **Authentication** → **Policies** en el dashboard de Supabase y comprueba que cada tabla tiene sus políticas listadas.

4.4. Crear el usuario administrador

El primer usuario con rol ADMIN debe crearse manualmente:

1. En Supabase, ve a **Authentication** → **Users**.
2. Pulsa **"Add User"** → **"Create new user"**.
3. Rellena:
   - **Email:** `admin@agrogestion.es` (o el email que desees)
   - **Password:** (mínimo 6 caracteres)
4. Marca ✅ **"Auto Confirm User"**.
5. Pulsa **"Create User"**.
6. Ve al **SQL Editor** y ejecuta:

```
-- Asignar rol ADMIN al usuario creado
UPDATE perfiles
SET id_rol = 1
WHERE email = 'admin@agrogestion.es';  -- Usa el email que elegiste
```

> ***Nota:*** *El trigger* `on_auth_user_created` *habrá creado automáticamente la fila en la tabla* `perfiles` *con nombre/apellidos/dni vacíos. Puedes completarlos con un UPDATE adicional si lo deseas.*

4.5. Obtener credenciales de conexión

1. En Supabase, ve a **Settings** → **API** (en el menú lateral).
2. Copia los siguientes valores:
   - **Project URL** (ej: `https://xxxxx.supabase.co`)
   - **anon public key** (una cadena larga que empieza por `eyJ...`)
3. Estos valores se usarán en el siguiente paso.

---

5. Configurar el entorno local

5.1. Instalar Node.js

1. Descarga Node.js desde https://nodejs.org (versión LTS recomendada).
2. Ejecuta el instalador y sigue los pasos (acepta los valores por defecto).

3. Reinicia el terminal.
4. Verifica la instalación:

```
node --version
npm --version
```

5.2. Instalar dependencias del proyecto

Desde el directorio raíz del proyecto (`AGROGESTION/`), ejecuta:

```
npm install
```

Este comando descargará todas las dependencias definidas en `package.json` . Puede tardar 1-3 minutos dependiendo de la conexión a Internet.

Dependencias principales que se instalarán:

Paquete	Función
react 19, react-dom 19	Librería de interfaz de usuario.
@supabase/supabase-js	Cliente para comunicar con Supabase.
tailwindcss 4	Framework CSS de utilidades (v4, sintaxis nueva).
@tailwindcss/vite	Plugin de TailwindCSS para Vite.
react-router-dom 7	Enrutamiento y navegación SPA.
i18next, react-i18next	Internacionalización (español, inglés y rumano).
zustand	Gestión de estado global ligera.
dotenv	Gestión de variables de entorno.
vite 7	Herramienta de build y servidor de desarrollo.
typescript 5.9	Compilador TypeScript.

5.3. Configurar variables de entorno

Crea un archivo llamado `.env` en la raíz de la carpeta `AGROGESTION/` con el siguiente contenido:

```
VITE_SUPABASE_URL=https://TU_PROYECTO.supabase.co
VITE_SUPABASE_ANON_KEY=TU_CLAVE_ANON_PUBLICA
```

Sustituye los valores por los que copiaste en el paso 4.4.

> ⚠️ ***IMPORTANTE:*** *El archivo `.env` contiene información sensible. NO lo subas a repositorios públicos. Ya está incluido en `.gitignore` .*

---

6. Ejecutar la aplicación en modo desarrollo

Desde la carpeta `AGROGESTION/`, ejecuta:

```
npm run dev
```

Verás un mensaje similar a:

```
  VITE v7.2.4  ready in 300ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: http://192.168.x.x:5173/
  ➜  press h + enter to show help
```

Abre tu navegador y accede a:

http://localhost:5173

La aplicación se recargará automáticamente (Hot Module Replacement) cada vez que guardes cambios en el código.

Para detener el servidor de desarrollo, pulsa `Ctrl + C` en el terminal.

---

7. Construir la aplicación para producción

Para generar los archivos optimizados para producción:

```
npm run build
```

Este comando:

1. Compila el código TypeScript.
2. Empaqueta y minifica todos los archivos (HTML, CSS, JavaScript).
3. Genera la carpeta `dist/` con los archivos listos para subir a un servidor.

Para previsualizar el build antes de subir:

```
npm run preview
```

Se abrirá un servidor local en `http://localhost:4173` con la versión de producción.

---

8. Desplegar en un servidor

8.1. Despliegue en Vercel (recomendado)

Vercel es la plataforma recomendada para proyectos Vite/React.

Pasos:

1. Crea una cuenta en https://vercel.com (puedes usar GitHub).
2. Pulsa **"Add New Project"**.
3. Selecciona el repositorio **Juanfrina/Agrogestion.Web** desde GitHub.
4. Configura el proyecto:
   - **Framework Preset:** Vite
   - **Root Directory:** `AGROGESTION`

- **Build Command:** `npm run build`
- **Output Directory:** `dist`

5. Añade las **variables de entorno**:
   - Clave: `VITE_SUPABASE_URL` → Valor: tu URL de Supabase
   - Clave: `VITE_SUPABASE_ANON_KEY` → Valor: tu anon key
6. Pulsa **"Deploy"**.
7. En 1-2 minutos tendrás la URL de producción (ej: `https://agrogestion.vercel.app`).

> ***Nota:*** *El proyecto incluye un archivo* `vercel.json` *con la configuración de reescritura SPA:*
>
> ```
> { "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
> ```
>
> *Esto garantiza que las rutas del lado del cliente (React Router) funcionen correctamente al recargar cualquier página.*

Despliegues posteriores: Cada push a la rama `main` en GitHub desplegará automáticamente la nueva versión.

8.2. Despliegue en Netlify

1. Crea una cuenta en https://netlify.com.
2. Pulsa **"Add new site"** → **"Import an existing project"**.
3. Conecta tu cuenta de GitHub y selecciona el repositorio.
4. Configura:
   - **Base directory:** `AGROGESTION`
   - **Build command:** `npm run build`
   - **Publish directory:** `AGROGESTION/dist`
5. Añade las variables de entorno en **Site settings** → **Environment variables**.
6. Pulsa **"Deploy site"**.

8.3. Despliegue en servidor propio (Apache/Nginx)

Paso 1: Construir la aplicación

```
cd AGROGESTION
npm install
npm run build
```

Paso 2: Subir los archivos

Copia el contenido de la carpeta `dist/` al directorio raíz del servidor web:

- **Apache:** `/var/www/html/agrogestion/`
- **Nginx:** `/usr/share/nginx/html/agrogestion/`

Puedes usar FTP, SCP o cualquier método de transferencia:

```
scp -r dist/* usuario@servidor:/var/www/html/agrogestion/
```

Paso 3: Configurar redirección SPA

Como Agrogestión es una SPA (Single Page Application), todas las rutas deben redirigirse a `index.html`.

Apache — Crea un archivo `.htaccess` en la carpeta del proyecto:

```
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteBase /
    RewriteRule ^index\.html$ - [L]
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteCond %{REQUEST_FILENAME} !-d
    RewriteRule . /index.html [L]
</IfModule>
```

Nginx — Añade al bloque `server`:

```
server {
    listen 80;
    server_name tu-dominio.com;
    root /usr/share/nginx/html/agrogestion;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Reinicia el servidor web tras los cambios:

```
# Apache
sudo systemctl restart apache2

# Nginx
sudo systemctl restart nginx
```

---

9. Verificación del despliegue

Una vez desplegada la aplicación, verifica lo siguiente:

Verificación	Cómo comprobar
La página carga correctamente	Accede a la URL y verifica que se muestra la landing page.
La conexión con Supabase funciona	Intenta registrarte o iniciar sesión.
El registro incluye selección de rol	Al registrarse, verifica que aparecen Gerente, Capataz y Trabajador.
Las rutas funcionan correctamente	Navega entre páginas y recarga (F5) en cada una.
El diseño responsive es correcto	Prueba en móvil o redimensiona el navegador.
El menú hamburguesa funciona en móvil	En pantalla pequeña, verifica que el sidebar se colapsa.

El cambio de idioma funciona	Cambia entre español, inglés y rumano desde la interfaz.
No hay errores en la consola del navegador	Abre DevTools (F12) y revisa la pestaña Console.
Las políticas RLS funcionan	Inicia sesión con distintos roles y verifica los datos visibles.
Las notificaciones se generan	Asigna una tarea a un capataz y verifica que recibe notificación.

---

10. Solución de problemas comunes

Error: "npm: command not found"

- **Causa:** Node.js no está instalado o no está en el PATH.
- **Solución:** Instala Node.js desde https://nodejs.org y reinicia el terminal.

Error: "VITE_SUPABASE_URL is undefined"

- **Causa:** El archivo `.env` no existe o las variables no tienen el prefijo `VITE_`.
- **Solución:** Verifica que el archivo `.env` existe en la carpeta `AGROGESTION/` y que las variables empiezan por `VITE_`.

Error: "Failed to fetch" al hacer login

- **Causa:** La URL de Supabase o la anon key son incorrectas.
- **Solución:** Verifica los valores en `.env` comparándolos con los de Settings → API en Supabase.

Error: "relation 'usuario' does not exist"

- **Causa:** Las tablas no se han creado o se usó un script antiguo que usa `usuario` en vez de `perfiles`.
- **Solución:** Ejecuta el script `schema.sql` actualizado (ubicado en `AGROGESTION/src/database/supabase/schema.sql`) en el SQL Editor de Supabase. La tabla de usuarios se llama `perfiles`.

La página se queda en blanco

- **Causa:** Error de JavaScript. Puede ser un problema de rutas o de build.
- **Solución:** Abre la consola del navegador (F12 → Console) para ver el error específico.

Error 404 al recargar una ruta (producción)

- **Causa:** El servidor no está configurado para redirigir a `index.html`.
- **Solución:** Configura la redirección SPA según el paso 8.3.

Error de CORS

- **Causa:** El dominio de producción no está en la lista de orígenes permitidos de Supabase.
- **Solución:** En Supabase → Settings → API → añade tu dominio en "Additional Redirect URLs".

npm install falla con errores de permisos

- **Causa:** Problemas de permisos en Windows o Linux.
- **Solución Windows:** Ejecuta el terminal como Administrador.
- **Solución Linux/Mac:** No uses `sudo npm install`. Configura npm para que no requiera permisos de root.

Resumen de comandos

Acción	Comando
Clonar repositorio	`git clone https://github.com/Juanfrina/Agrogestion.Web.git`
Entrar en la carpeta	`cd Agrogestion.Web/AGROGESTION`
Instalar dependencias	`npm install`
Ejecutar en desarrollo	`npm run dev`
Construir para producción	`npm run build`
Previsualizar build	`npm run preview`
Verificar versión de Node.js	`node --version`
Verificar versión de npm	`npm --version`